

CSCI 232 Assignment 05 – Linked List in C++ & C/C++ Integration

Points: 10

Instructions

1. **Fork** the repository on GitHub: <https://github.com/jpach-cs/232-ass05-template>
2. **Clone** your forked repository to your local machine.
3. Open `src/code.cpp` and `src/tests.cpp` and `include/code.hpp`.
4. Uncomment the author block at the top of `src/code.cpp` and fill in your name:

```
// std::string AUTHOR_NAME      = "Your Name";  
// std::string AUTHOR_AUTHSHIP = "I acknowledge that I have worked on this  
// assignment independently...";
```

5. Work through each function and class method in **two parts**:

Part 1 — Legacy Data & Node Integration (`src/code.cpp`)

- Implement `printLegacyData` to safely handle legacy C unions in C++.
- Implement node initialization and helper creation functions for `struct`, `classNodeVariant`, and template nodes.
- Do **not** modify function signatures in `include/code.hpp`.

Part 2 — LinkedList Class Implementation (`src/code.cpp`)

- Implement RAII memory management in the constructor and destructor.
- Implement member methods for insertion, deletion, searching, and traversal.
- Ensure all methods handle `nullptr` guards and empty list edge cases.

Part 3 — Unit Tests (`src/tests.cpp`)

- Implement unit test bodies using `TEST_ASSERT_`.
- Clean up any dynamically allocated memory in test cases to prevent leaks.

1. Build and run locally using C++20 (g++):

```
make clean  
make  
./run_tests
```

7. When all local tests pass — **commit** and **push** to your repository.
 8. Go to the **Actions** tab on GitHub and verify the CI results.
-

Functions — `src/code.cpp` (Part 1)

Function	Description
<code>printLegacyData</code>	Format <code>LegacyData</code> union value (<code>int</code> , <code>double</code> , <code>char*</code>) into <code>std::string</code>
<code>initStructNode</code>	Initialize a raw <code>StructNode</code> with a value and clear <code>next</code>
<code>createTwoStructNodes</code>	Allocate and w-link two <code>StructNode</code> instances on the heap
<code>createTwoClassNodes</code>	Allocate and w-link two <code>classNodeVariant</code> objects on the heap
<code>createTwoTemplateNodes</code>	Instantiate and link two generic <code>TemplateNode<T></code> objects

LinkedList Class Methods — `src/code.cpp` (Part 2)

Method	Description
<code>LinkedList()</code>	Constructor — initialize empty list (<code>head = nullptr</code> , <code>count = 0</code>)
<code>~LinkedList()</code>	Destructor — free all allocated nodes in the list (RAII)
<code>addFirst</code>	Insert <code>classNodeVariant*</code> at the front of the list (with <code>nullptr</code> guard)
<code>addLast</code>	Insert <code>classNodeVariant*</code> at the end of the list (with <code>nullptr</code> guard)
<code>deleteFirst</code>	Remove and free the first node; update <code>head</code> and <code>count</code>
<code>deleteLast</code>	Remove and free the last node; update pointers and <code>count</code>
<code>deleteValue</code>	Search, remove, and free the first node matching <code>NodeValue</code>
<code>printList</code>	Print all node values to <code>std::cout</code> using <code>std::visit</code> ; return count
<code>listLength</code>	Return the current number of nodes in the list

Tests to implement — `src/tests0.cpp`

Test function	What to verify
<code>test_printLegacyData_int</code>	Formats integer field 'i' into valid <code>std::string</code>
<code>test_printLegacyData_double</code>	Formats double field 'd' rounded to 2 decimal places
<code>test_printLegacyData_string</code>	Formats <code>char*</code> string or returns <code>"nullptr"</code> if NULL
<code>test_printLegacyData_null_ptr</code>	Safely handles NULL string pointer without crashing
<code>test_printLegacyData_unknown_type</code>	Returns <code>"unknown"</code> for unsupported type char
<code>test_initStructNode_sets_fields</code>	Sets value correctly and sets <code>next</code> to <code>nullptr</code>
<code>test_linkedList_addFirst_updates_count</code>	<code>addFirst</code> updates <code>head</code> and increments <code>count</code>
<code>test_linkedList_addFirst_null_guard</code>	<code>addFirst(nullptr)</code> does not increment counter or crash
<code>test_linkedList_addLast_null_guard</code>	<code>addLast(nullptr)</code> handles NULL input safely
<code>test_linkedList_deleteFirst_empty</code>	<code>deleteFirst()</code> on empty list does not crash
<code>test_linkedList_deleteLast_empty</code>	<code>deleteLast()</code> on empty list handles edge case safely
<code>test_linkedList_deleteValue_not_found</code>	<code>deleteValue()</code> with missing value leaves list unchanged
<code>test_linkedList_printList_empty</code>	<code>printList()</code> on empty list returns 0

Rules

- Do **not** modify headers or function **signatures** in `inc/code.hpp`.
- All code must compile under `C++20` (`-std=c++20`).
- Always guard against `nullptr` when dereferencing pointers.
- Always clean up dynamically allocated nodes (`delete`) in tests and destructor.
- CI runs your `tests05.cpp` together with instructor validation tests — both must pass.

Grading

Component	Points
Legacy integration & node creation functions	3
<code>LinkedList</code> class implementation (RAII & methods)	4
All test cases in <code>tests05.cpp</code> passing	3
Total	10

Author block — required

```
std::string AUTHOR_NAME      = "Your Name";
std::string AUTHOR_AUTHSHIP = "I acknowledge that I have worked on this "
"assignment independently, except where explicitly noted and referenced. "
"Any collaboration or use of external resources has been properly cited. "
"I am fully aware of the consequences of academic dishonesty and agree "
"to abide by the university's academic integrity policy.";
```